

AetherQuiz AI

PDF-to-Interactive Quiz Generator

Document Class: Technical & Deployment Blueprint

Target Environment: University LMS (Canvas, Moodle, Blackboard)

Hosting Stack: 100% Free Production Deployment (Cloudflare + Render)

AI Engine: Groq Llama-3.3-70B API Tier

Prepared For: Course Tutors and LMS Administrators

Date: July 2026

Status: Production Ready

1. Executive Summary & Core Concept

AetherQuiz AI is a lightweight, high-performance web application designed for university instructors to rapidly translate course materials (syllabi, reading packets, slides) into structured assessment questions. By pairing a high-fidelity PDF parser with LLM completion, the system automates quiz formulation and distractor generation.

The system operates in two core modes:

- **Auto-Gen Mode:** Instructors specify a desired number of questions. The system scans the document, identifies key conceptual topics, and instructs the LLM to write corresponding multiple-choice questions.
- **Custom Prompts Mode:** Instructors input specific question stems or topics. The system reads the PDF, locates the factual answers, creates three plausible distractor options, and formats the output into standard A-D configurations.

2. System Architecture & Tech Stack

To ensure zero cost barrier and maximum reliability, AetherQuiz AI uses a bifurcated serverless-static cloud architecture:

Component	Technology	Role	Hosting / Deployment
Frontend UI	React 18 + Vite + CSS	User dashboard, editor, interactive practice	Cloudflare Pages (Static)
Backend API	FastAPI (Python 3.10+)	Saves uploaded PDF, extracts text, handles prompts	Render.com (Web Service)
PDF Parser	PyMuPDF (fitz)	Fast, clean local extraction of text from PDF binaries	Embedded in Backend
AI Inference	Groq Llama-3.3-70B	Receives system prompts, outputs structured JSON quiz results	OpenAI API call

3. The 100% Free Production Stack Strategy

AetherQuiz AI is built to run entirely on zero-cost tiers, eliminating hosting fees. The breakdown below describes how each layer achieves this, along with specific limitations.

Frontend: Cloudflare Pages (Free Tier)

Cloudflare Pages hosts the pre-compiled static assets (HTML, CSS, JS). The free tier provides unlimited bandwidth, unlimited requests, and 500 builds per month. Pages are served directly from Cloudflare's global edge network, meaning pages load instantly with 0ms cold starts.

Backend: Render.com Free Web Service

The Python backend runs on Render's *Free Web Service* tier. This tier gives you 512MB RAM, shared CPU, and 500 free build hours per month (which is plenty for low-to-medium volume tutoring tools). The primary drawback is **cold starts**: if the API has not received any traffic for 15 minutes, Render spins down the server container. The next request will experience a 50-second spin-up delay. Once spun up, it remains active as long as requests are ongoing.

AI Layer: Groq Developer API (Free Tier)

Groq runs LLM inference at extremely high speeds. The Developer Tier is free, requires no credit card, and has limits based on rate and token quotas instead of pricing. This makes it perfect for deployment in classrooms.

4. Groq API Quotas, Token Limits & Refresh Windows

Groq manages API usage using a combination of rate limits (number of hits) and token limits (amount of text processed). The system defaults to the **llama-3.3-70b-versatile** model, which falls under the following developer limits:

Limit Metric	Value / Quota	Window Duration	How it Refreshes
RPM (Requests Per Min)	30 requests	1 Minute (Sliding)	Refreshes continuously per-second (1 req / 2s avg)
RPD (Requests Per Day)	14,400 requests	24 Hours (Fixed)	Resets at midnight UTC (00:00 UTC) every day
TPM (Tokens Per Min)	40,000 tokens	1 Minute (Sliding)	Tokens from completed requests age out after 60s
TPD (Tokens Per Day)	500,000 tokens	24 Hours (Fixed)	Resets at midnight UTC (00:00 UTC) every day

Understanding Token Refresh Mechanics

- 1. Sliding-Window Limits (TPM / RPM):** Rather than resetting completely on the hour, sliding window tokens refresh progressively. For instance, if you submit a request with 8,000 tokens at 12:01:05, those 8,000 tokens are deducted from your 40,000 TPM limit. At 12:02:05, those 8,000 tokens are returned to your pool.
- 2. Daily Fixed Limits (TPD / RPD):** Unlike sliding limits, the daily caps are absolute counters that count up to 500,000 tokens and reset all at once at 00:00 UTC. If you hit 500,000 tokens at 15:00 UTC, the API will return HTTP 429 error codes until 00:00 UTC.

5. Token Consumption & Cost Calculations

AetherQuiz AI contains built-in optimization to keep usage within the free-tier limits. In **main.py**, the extracted PDF text is capped using python slice notation before sending it to the model:

```
trimmed = text[:12000] # Cap input text context to preserve tokens
```

12,000 characters correspond roughly to **2,000 to 2,500 words** or approximately **3,000 tokens**. This is the maximum input size possible.

Worst-Case Request Math (10-Question Quiz):

- **Input Context:** Capped PDF Text + Instruction Prompt = ~3,200 input tokens.
- **Output Response:** Generated 10-Question JSON Quiz = ~1,200 output tokens.
- **Total Tokens Per Request:** ~4,400 tokens.

Given these average values, your limits yield the following capacities:

- **Max Quizzes Per Minute:** 40,000 TPM / 4,400 tokens = ~9 quizzes per minute (exceeds standard classroom usage).
- **Max Quizzes Per Day:** 500,000 TPD / 4,400 tokens = ~113 quizzes generated per day.

6. Project Guide & Deployment Blueprint

Follow these structured instructions to deploy the backend and frontend services to production.

Step A: Render Backend Web Service Setup

1. Register a free account at **render.com** and connect your GitHub repository.
2. Select **New + > Web Service**.
3. Configure the settings exactly as follows:
 - **Runtime:** Python
 - **Build Command:** `pip install -r requirements.txt`
 - **Start Command:** `uvicorn main:app --host 0.0.0.0 --port $PORT`
 - **Root Directory:** Leave blank
4. Navigate to the **Environment** tab and add your Groq key:
 - **Key:** `GROQ_API_KEY`
 - **Value:** [Your API key beginning with `gsk_`]
5. Click **Deploy**. Render will build and deploy. Take note of the live URL (e.g. `https://your-app.onrender.com`).

Step B: Cloudflare Pages Frontend Setup

1. The frontend is located in the **frontend/** directory. Navigate to this folder on your machine.
2. Install local node dependencies and run the build compiler:

```
cd frontend npm install npm run build
```

This generates a highly optimized distribution bundle in the **frontend/dist** directory.

3. Deploy the compiled build straight to Cloudflare's edge network using Wrangler:

```
npx wrangler pages deploy dist
```

4. Wrangler will ask you to log in to Cloudflare. Once completed, select your project name and it will upload the files.
5. Note down the Pages live URL (e.g., `https://aether-quiz.pages.dev`).

7. Troubleshooting & API Connection

Once both systems are deployed, you must bridge the two URLs in your user dashboard:

1. Open your live Cloudflare Pages URL in a browser.
2. Look for the **API Configuration / Endpoint** field in the dashboard UI settings (usually in the header or sidebar).
3. Paste your full Render live URL (e.g., `https://tutor-qa-deployment-test.onrender.com`).
4. **Cold Start Warning:** If the backend has been idle and you request a quiz, it will say 'Connecting...' for 45-60 seconds. This is normal behavior for Render's free tier. You can keep the server awake by using a free cron monitoring service (like UptimeRobot) to ping the health endpoint (`/`) every 14 minutes.